

WHAT I LEARNED FROM ASSIGNMENT 8

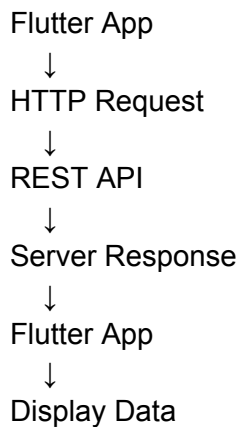
REST API & LOCAL CACHING IN FLUTTER

1. REST API INTEGRATION

In this assignment, I learned how a Flutter application can communicate with an external REST API.

I learned how to send an HTTP GET request using the http package and receive data from a remote server. Instead of keeping all application data inside the Flutter code, the application can retrieve dynamic information from an external API.

I also understood the basic API workflow:



2. WORKING WITH JSON DATA

I learned that APIs commonly return data in JSON format.

The received JSON data cannot always be used directly in the UI, so it needs to be decoded and converted into a suitable Dart structure.

In this project, I created a Post model containing properties such as:

- id
- userId
- title
- body

The JSON response is converted into Post objects using a factory constructor.

This helped me understand how real applications transform external API data into structured application data.

3. DART MODELS AND JSON PARSING

I learned why creating a model class is better than directly working with dynamic JSON throughout the application.

The Post model provides a clear structure for the data and makes the rest of the application easier to understand.

The process I learned was:

JSON Response



jsonDecode()



Post.fromJson()



Post Objects



UI

I also learned how `toJson()` can convert a Dart object back into JSON format, which is useful when storing or sending data.

4. ASYNCHRONOUS PROGRAMMING

One of the important concepts I practiced was asynchronous programming.

Network requests do not complete immediately, so Flutter cannot simply wait without affecting the application.

I learned how to use:

- Future
- async
- await
- try-catch

Using these concepts, the application can perform network operations while keeping the user interface responsive.

This improved my understanding of how Flutter handles operations that take time, such as API requests and local storage operations.

5. API ERROR HANDLING

I learned that an API request can fail for many reasons, such as:

- No internet connection
- Server error
- Request timeout
- Invalid response
- Unexpected JSON structure
- Empty API response

Instead of allowing these problems to crash the application, I implemented error handling using try-catch and response validation.

This taught me that handling failure cases is an important part of developing a reliable application.

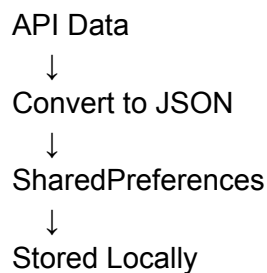
6. LOCAL STORAGE WITH SHARED PREFERENCES

Another major concept I learned was local data storage.

I used the SharedPreferences package to store API data locally on the device.

When the API successfully returns data, the application saves that information in local storage.

The process is:



This helped me understand how applications can preserve useful information locally instead of depending completely on the network.

7. CACHING

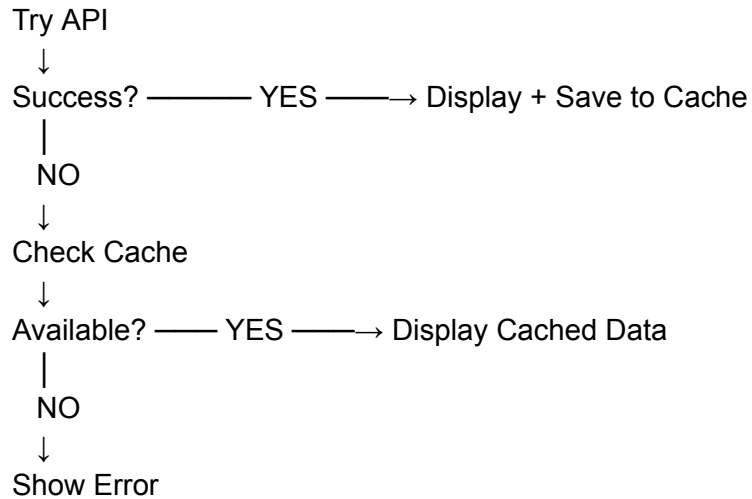
Through this assignment, I learned the practical purpose of caching.

Caching means keeping a previously retrieved copy of data so that it can be reused later.

In PulseAPI, when fresh API data is received, it is saved to the local cache.

If the API is unavailable, the application checks the cache and displays the previously stored data.

The complete flow is:



This taught me how caching can make an application more reliable and useful during network problems.

8. CACHE TIMESTAMP

I also learned that cached data can have additional information associated with it.

In this project, the time when the cache was updated is stored along with the cached posts.

The application can then determine how old the cached data is.

For example:

- Cached just now
- Cached 5 minutes ago
- Cached 2 hours ago

This gave me an understanding of how real applications can provide information about the freshness of locally stored data.

9. SEPARATION OF RESPONSIBILITIES

This assignment helped me understand the importance of organizing code into different responsibilities.

The project separates different tasks into different files:

Post Model

→ Handles the structure of data.

ApiService

→ Handles communication with the REST API.

CacheService

→ Handles local data storage and retrieval.

HomeScreen

→ Handles the user interface.

This separation makes the application easier to understand, maintain and modify.

I learned that keeping API logic, storage logic and UI logic separate is much better than writing everything inside one screen.

10. DYNAMIC UI WITH FUTUREBUILDER

I learned how Flutter can display different UI states depending on the result of an asynchronous operation.

The application handles states such as:

- Loading
- Success
- Empty data
- Error

This is important because an API-driven application does not always have data immediately available.

The UI can therefore respond dynamically instead of showing a blank screen while the request is running.

11. RESPONSIVE UI DESIGN

I also improved my understanding of responsive design.

The PulseAPI dashboard was designed to work across different screen sizes, including:

- Desktop
- Tablet
- Mobile

The layout changes according to the available screen width so that the content remains readable and usable.

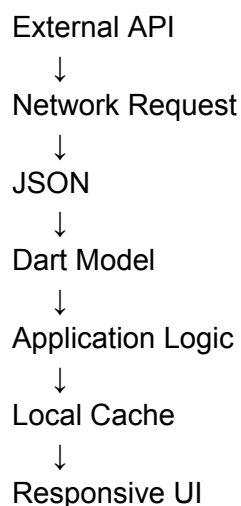
This taught me that Flutter applications should not be designed only for one fixed screen size.

12. REAL-WORLD APPLICATION THINKING

The biggest lesson from this assignment was understanding how a real application works with external data.

Previously, an application could display static information directly from the code.

In this assignment, the data flow became:



This helped me understand the connection between frontend UI, backend APIs, network communication and local storage.

13. OVERALL LEARNING

Assignment 8 gave me practical experience in building a Flutter application that works with real external data.

The main concepts I learned were:

- REST API integration
- HTTP GET requests
- JSON decoding
- Dart model classes
- Future and async/await
- API error handling
- SharedPreferences

- Local caching
- Cache timestamps
- Offline fallback
- FutureBuilder
- Dynamic UI states
- Responsive Flutter layouts
- Separation of responsibilities

Overall, this assignment helped me move beyond static Flutter applications and understand how to build applications that communicate with external APIs, handle network failures and maintain useful functionality through local cached data.